

# Sobre o módulo

terça-feira, 9 de maio de 2023 20:31

Neste módulo você será capaz de identificar, definir e diferenciar os conceitos básicos de orientação a objetos, tanto em teoria quanto em Java: classes, objetos, atributos de classes, construtores de classes, responsabilidades, colaborações e cartões CRC

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/home/week/1>>

# Conhecendo as Classes

terça-feira, 4 de abril de 2023 15:52

# Identificando Classes e Objetos

terça-feira, 4 de abril de 2023 10:39

Pensando em um sistema de pet-shop, podemos ter:

- Cão
- Produto
- Usuário

Estas são as **Entidades**;

**Entidades** colaboram para criar a funcionalidade do software.

As **entidades** que colaboram na criação de um sistema são as suas **classes**!

A classe é uma **abstração**.

**Objeto**: Instância Concreta da classe.

**Classe Cadeira** cria/instância **Objetos Cadeira**.

**Objetos são únicos!**

- Eu posso ter 2 cadeiras iguais, mas são objetos diferentes.

# Comportamento e Estado das Classes

terça-feira, 4 de abril de 2023 11:16

As características de um objeto da classe são chamados de **atributos**!

A classe também pode realizar ações!

Uma ação pode inclusive, mudar valores de **atributos** de um objeto!

As ações que um objeto da classe pode realizar são representados por **métodos**!

**Estado:** Atributos

**Comportamento:** Métodos

**Classe:**

Abstrato

Tem Atributos

Tem ações

**Objeto:**

Concreto

Tem Valores

Executa ações

# Conhecendo as Classes com Java

terça-feira, 4 de abril de 2023

16:46

# Criando classes com Java

terça-feira, 4 de abril de 2023 15:53

Foi criado uma classe em Java

# Construtores de Classes

terça-feira, 4 de abril de 2023 16:48

**Construtores** são "métodos especiais" usados para criar objetos da classe. Com eles você pode parametrizar o objeto criado e inicializar variáveis!

**this**: usado para referenciar elementos da classe.

Foram criados exemplos práticos em Java

# CRC: Classe, Responsabilidade e Colaboração

terça-feira, 4 de abril de 2023 18:06



# Identificando Responsabilidades

terça-feira, 4 de abril de 2023 18:06

Antropomorfismo:

Capacidade de aplicar conceitos e características do ser humano, em objetos inanimados.

Em OO:

Aplicar a objetos e classes os conceitos e comportamentos próprios de um ser humano.

- **Homem**

**Sabe:** seu nome, data de nascimento, endereço...

**Faz:** andar, falar, pular...

- **Cachorro**

**Sabe:** seu nome, onde é sua casa, quem é seu dono...

**Faz:** andar, latir, correr...

## Responsabilidade

O que uma classe **sabe** ou **faz**, é chamado de **Responsabilidade**.

Os **atributos** e **métodos** de uma classe juntos, são as **Responsabilidades** da Classe.

- **Sabe:** são os **atributos**

- **Faz:** são os **métodos**

# Identificando a Lógica das Responsabilidades

terça-feira, 4 de abril de 2023 18:42

## Classe Conta Corrente

Responsabilidades:

- **Sabe:**
  - o número da conta
  - o valor do saldo atual
- **Faz:**
  - credita valor ao saldo atual
  - debita valor do saldo atual
  - é possível retirar determinado valor?

Qual é a lógica da responsabilidade "**Sabe o número da conta**"?

- Objeto conta tem o atributo número da conta
- Objeto devolve o valor do número da conta

Qual é a lógica da responsabilidade "**é possível retirar determinado valor**"?

- Se saldo atual > valor
- Então **Sim**
- Se não **Não**

O objetivo instrucional era entender que uma responsabilidade corresponde a "o quê" deve ser feito e que a lógica da responsabilidade corresponde a "como" deve ser feito. Por exemplo, a lógica de uma responsabilidade do tipo faz corresponde ao conjunto de ações que a responsabilidade deve realizar.

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/6HNPo/identificando-a-logica-das-responsabilidades>>

## Classe Banco

Responsabilidades:

- **Sabe:**
  - As contas ativas - **Atributo:** **lista de objetos de contas ativas**
  - As contas inativas - **Atributo:** **lista de objetos de contas inativas**
- **Faz:**
  - Registra uma conta nova
  - Apresenta os números das contas ativas
  - Obtém o saldo total do banco

Qual é a lógica da responsabilidade "**Registra uma conta nova**"?

- Cria um objeto conta corrente novo
- Insere esse objeto na **lista de objetos contas ativas**

Qual é a lógica da responsabilidade "**Apresenta os números das contas ativas**"?

- O objeto Banco tem o atributo **lista de contas correntes ativas** do Banco
- Cria uma lista vazia de números de contas

Para cada objeto **conta corrente (ativa)** da lista de contas ativas:

- O objeto conta corrente informa o seu nº de conta, pela responsabilidade "**sabe o número da conta**"
- Inclui o nº da conta na lista de números de contas
- Passa para outra conta corrente se houver

Por fim:

- Apresenta os números da lista de números de contas

Qual é a lógica da responsabilidade "**Obtém saldo total do banco**"?

- O objeto Banco tem o atributo **lista de contas correntes ativas** do Banco
- Cria uma variável temporária saldo, zerada

Para cada objeto **conta corrente (ativa)** da lista de contas ativas:

- O objeto conta corrente informa o seu saldo, pela responsabilidade "**sabe o valor do saldo atual**"
- Saldo = saldo anterior + saldo da conta corrente
- Passa para a outra conta corrente se tiver

Por fim:

- Devolve o valor de saldo, que é o saldo total do banco

Para realizar a lógica de uma responsabilidade, eu tenho duas ações:

- Um objeto da classe pode realizar a tarefa sozinho ou
- Solicitar a colaboração de objetos de **classe colaboradora**

# Identificando Colaborações

quarta-feira, 5 de abril de 2023 11:04

Para explicar melhor sobre **colaborações**, aqui a Lógica da Responsabilidade "**Apresenta os números das contas ativas**":

- O objeto Banco tem o atributo **lista de contas correntes ativas** do Banco
- Cria uma lista vazia de números de contas

Para cada objeto **conta corrente (ativa)** da lista de contas ativas:

- O objeto conta corrente informa o seu nº de conta, pela responsabilidade "**sabe o número da conta**"
- Inclui o nº da conta na lista de números de contas
- Passa para outra conta corrente se houver

Por fim:

- Apresenta os números da lista de números de contas

Sendo assim:

O objeto **Banco** tem uma **lista de contas correntes**, **objetos** conta corrente.

Pra pegar os números destas contas, eu preciso examinar cada conta da **lista de objetos conta corrente**.

Para cada uma das contas, eu peço para a **Conta Corrente** o nº da conta, pois a **Conta Corrente** **sabe** o nº.

A lógica da responsabilidade seria: estou pedindo para um objeto da classe **Conta Corrente**, prestar um serviço pra mim, ou seja, estou pedindo uma **Colaboração**.

Neste caso a classe **Conta Corrente** é uma **Classe Colaboradora**.

Estou utilizando a responsabilidade **sabe o nº** da classe **Conta Corrente**.

**Uma colaboração é uma responsabilidade.**

**As tabelas modeladas desta forma são chamadas de Cartão CRC.**

|                               |                    |
|-------------------------------|--------------------|
| Classe: <b>Conta Corrente</b> |                    |
| <b>Responsabilidade</b>       | <b>Colaboração</b> |
| Credita valor ao saldo atual  |                    |
| Debita valor do saldo atual   |                    |

|                                    |                                                                          |
|------------------------------------|--------------------------------------------------------------------------|
| Classe: <b>Banco</b>               |                                                                          |
| <b>Responsabilidade</b>            | <b>Colaboração</b>                                                       |
| Registra uma conta nova            | Colaboradora: Conta Corrente<br>Colaboração: Construtor                  |
| Apresenta os nrs das contas ativas | Colaboradora: Conta Corrente<br>Colaboração: Sabe o nº da conta          |
| Obtém o saldo total do banco       | Colaboradora: Conta Corrente<br>Colaboração: Sabe o valor do saldo atual |

**Colaboração:** Corresponde a uma responsabilidade da classe colaborada.

**Classe Colaboradora:** classe servidora. (oferece serviços a outras classes)

**Classe que depende de classe colaboradora:** classe cliente.

Classe Cliente -----> Classe Servidora: A classe cliente depende da classe servidora.

Toda vez que uma **responsabilidade** minha precisa de uma **colaboração** de uma classe colaboradora, existe o **acoplamento** entre estas duas classes.

Dependência e Acoplamento significam a mesma coisa.

O objetivo é sempre diminuir o acoplamento entre classes.

# Cartão CRC

quarta-feira, 5 de abril de 2023

15:26

CRC = Classe Responsabilidade Colaboração

|                   |              |
|-------------------|--------------|
| Nome da Classe    |              |
| Responsabilidades | Colaborações |
|                   |              |
|                   |              |

|                                    |                                                                          |
|------------------------------------|--------------------------------------------------------------------------|
| Classe: <b>Banco</b>               |                                                                          |
| Responsabilidade                   | Colaboração                                                              |
| Registra uma conta nova            | Colaboradora: Conta Corrente<br>Colaboração: Construtor                  |
| Apresenta os nrs das contas ativas | Colaboradora: Conta Corrente<br>Colaboração: Sabe o nº da conta          |
| Obtém o saldo total do banco       | Colaboradora: Conta Corrente<br>Colaboração: Sabe o valor do saldo atual |

- Ajudar a explorar de maneira informal classes e objetos
- Fornece uma introdução mais fácil para orientação a objetos
- Usado largamente na aprendizagem de orientação a objetos
- Usado na indústria
- Ponto inicial de muitas metodologias orientadas a objetos
- Facilita a identificação e caracterização das classes

# Sobre o módulo

terça-feira, 9 de maio de 2023

20:33

Olá! Bem-vindo à semana 2 do curso Orientação a Objetos com Java! Nesta semana você aprofundará seu contato com classes e métodos, bem como com modelagem CRC. Ao final desta semana, você será capaz de: 1) modelar o comportamento de classes com métodos em Java; 2) projetar novas classes para uma aplicação por meio da modelagem CRC

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/home/week/2>>

# Lesson 1: Aprofundando nas classes com Java

quarta-feira, 5 de abril de 2023

15:46

# Relacionamento entre Classes

quarta-feira, 5 de abril de 2023 15:47

**As classes de um sistema devem ter responsabilidades bem definidas e colaborar para implementar as funcionalidades do sistema.**

Imagine uma pizzeria on-line em que o preço da pizza depende dos seus ingredientes e a entrega dependa do dia da semana e da distância.

## **Classe Pizza**

Calcula o preço baseado nos ingredientes da pizza.

## **Classe Entrega**

Calcula a entrega baseado no dia da semana e distância.

## **Classe Carrinho**

Calcula o valor total da compra com as pizzas e a entrega.

Exemplo:

O **Carrinho** "perguntaria" pra **Pizza** qual é seu preço, pro **Carrinho** não interessa como a **Pizza** calcula o total, calcular o preço de acordo com os ingredientes é responsabilidade da **Pizza**.

Criar um sistema orientado a objetos tem a ver com descobrir suas peças e como elas se encaixam!

# Hands-on: Colaborações entre Classes

quarta-feira, 5 de abril de 2023

19:29

Foi criada uma classe que registra pontos do usuário.

A ideia é que para este registro de pontuação exista um bônus, e pra este bônus, por exemplo, podem existir multiplicadores dependendo do tipo do usuário, ou fatores do dia...



# Métodos e Atributos Estáticos

quinta-feira, 6 de abril de 2023 17:11

## Classe

Algumas classes possuem variáveis que são compartilhadas com todas as instâncias.

## Instância

Instâncias possuem valores que são diferentes para cada uma delas.

```
public class Gato {  
    static int totalGatos = 0; // O mesmo valor é compartilhado por qualquer instância  
  
    Gato() {  
        totalGatos++; // Cada gato criado, incrementa +1  
    }  
}
```

Variáveis estáticas não varia de objeto para objeto, é da Classe.

A partir de qualquer objeto que eu tentar acessar totalGatos, irá acessar a mesma variável, pq ela é estática, é uma variável da classe.

**Gato.totalGatos;**

Não precisa de uma instância para acessar um membro estático.

**Não use variáveis estáticas como variáveis globais!**

Variáveis estáticas tem que ser utilizadas dentro do escopo de uma Classe, pertinentes a essa classe.

```
public Class Calc {  
    static int quadr(int i) {  
        return i*i;  
    }  
}
```

**Métodos estáticos são similares a funções na programação estruturada.**

```
acelerar(carro); // Pensamento estruturado  
carro.acelerar(); // Pensamento orientado a objetos
```

Os métodos static ou métodos da classe são funções que não dependem de nenhuma variável de instância, quando invocados executam uma função sem a dependência do conteúdo de um objeto ou a execução da instância de uma classe

De <<https://www.google.com/search?client=opera-gx&q=m%C3%A9todo+estat%C3%ADstico+java&sourceid=opera&ie=UTF-8&oe=UTF-8>>

# Hands-on: Comparando Tipos de Atributos – Estático x Instância

terça-feira, 11 de abril de 2023 11:23

Foi criado um projeto java, para demonstrar as diferenças nos tipos de variáveis.

Foi criado a Classe **Somador**, com 2 tipos de variáveis, 1 estática e 1 de instância. Toda vez que somar, soma 1 nas duas variáveis.

```
/*  
* A variável de instância é somada somente para aquele objeto  
* A variável estática esta somando para Todos os objetos  
* estática = todos objetos da classe acessam a mesma variável  
* instância = uma para cada objeto da classe  
*/
```

# Pensando em Métodos Orientados a Objetos

terça-feira, 11 de abril de 2023

11:59

- Criar métodos para classes que seguem ideias para uma modelagem orientada a objetos

Faz sentido termos métodos similares em classes diferentes? **Sim.**

- Faz sentido, quando o contexto é diferente.
- Uma classe Aluno pode ter um método para calcular a nota.
- Uma classe Turma pode ter um método de mesmo nome, mas para um contexto diferente.
- Nesse caso, o método de Turma chamaria o método de cada aluno.

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/0EqCl/pensando-em-metodos-orientados-a-objetos>>

Sempre que pensar em **métodos de objeto**, você tem que pensar:

- **Que informações fazem parte do escopo do objeto?**
- **Que informações não fazem e que são necessárias?**, para que assim **sejam passadas como parâmetro**

**Avalie que dados fazem parte do escopo da classe e quais serão recebidos como parâmetro pelos métodos!**

Não force colocar uma informação dentro da classe, que não seja pertinente a ela. Ex: **endereço** na classe **Carrinho**. O que for de fora, passar como parâmetro.

# Hands-on: Refatorando – de Funções para Métodos

terça-feira, 11 de abril de 2023

12:22

A ideia é mostrar um pouco mais como funciona a orientação a objetos, como são criados métodos orientados a objetos.

- Primeiro será criado um método com programação estruturada
- Depois será criado o mesmo método porém mais orientado a objetos

Na classe **VerificadoraNotas** serão criados métodos estáticos que equivalem a funções em programação estruturada.

A média do aluno é uma funcionalidade que tem a ver com o **Aluno**, então eu não deveria perguntar a outra classe qual a média desse aluno, deveria ser perguntado ao próprio **Aluno**.

Na abordagem orientada a objetos, o método **mediaAluno** estaria na própria classe **Aluno** e não na classe **VerificadoraNotas**.

Na programação estruturada, eram criadas as classes como estrutura de dados, e uma série de funções, que utilizam esta estrutura de dados. Sendo que muitas destas coisas, deveriam estar na classe.

# Lesson 2: Modelagem CRC

terça-feira, 11 de abril de 2023

13:55

# Modelagem CRC: Identificando Classes

terça-feira, 11 de abril de 2023 13:55

## CRC Modeling:

Uma abordagem informal para modelar uma aplicação com classes e objetos.

### Passo 0 - Especifica o Sistema

- **Sistema de automação da biblioteca:**
- A biblioteca tem um nome, mantém um catálogo de livros, onde cada livro tem título, autor, e número único de catálogo. Há Usuários da Biblioteca registrados, cada ...

### Passo 1 - Identifica Objetos e Classes

- Procurar por **substantivos/nomes** na especificação de requisitos do sistema pois podem ser **potencias objetos e classes**.
- **Substantivos:**
- A **biblioteca** tem um **nome**, mantém um **catálogo de livros**, onde cada **livro** tem **título**, **autor**, e **número único de catálogo**. Há **Usuários da Biblioteca** registrados, cada ...

### Passo 2 - Refina a lista de classes

- Identificar nomes diferentes que representam a mesma classe
- Retirar nomes que representam atributos
- Identificar classes e subclasses. Ex: **Soldado** e **Pessoa**, o soldado é uma pessoa, então **Soldado** seria a subclasse de **Pessoa**
- Descrever o que cada classe faz

obs: nome de classe é no **singular**

obs: quem está de fora do sistema é o **Ator**, que no exemplo seria a bibliotecária

- Descrição das classes:
- **Biblioteca:** representa o SAB (Sistema de Automação da Biblioteca)
- **Livro:** representa livros a serem emprestados a usuários da Biblioteca
- **Usuário:** representa usuários que emprestam livros da Biblioteca

Classes abstraídas do texto: **Biblioteca, Livro, Usuário**

# Modelagem CRC: Identificando Responsabilidades e Colaborações

terça-feira, 11 de abril de 2023

19:41

## Passo 3 - Responsabilidades Óbvias

- Identificar responsabilidade óbvias para cada classe

### Classe **Biblioteca**:

Sabe nome  
Sabe catálogo de livros  
Sabe lista de usuários  
Registra novo usuário  
Adiciona novo livro ao catálogo

### Classe **Livro**:

Sabe título  
Sabe autor  
Sabe número único de livro  
Sabe disponibilidade de empréstimo  
Sabe usuário, se emprestado

### Classe **Usuario**:

Sabe nome  
Sabe lista de livros emprestados (inicialmente vazia)

## Passo 4 - Verbos como responsabilidades

- Sistema de automação da biblioteca:
- A biblioteca tem um nome... Um usuário da Biblioteca pode emprestar um livro e devolver o livro...
- Bibliotecária tem que registrar um novo usuário...

Podemos identificar potenciais responsabilidades de classes examinando os **verbos de ação** encontrados na especificação de requisitos do sistema!

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/nCK5Q/modelagem-crc-identificando-responsabilidades-e-colaboracoes>>

## Passo 5 - Atribuição das novas responsabilidades

Para cada potencial responsabilidade, verifique a classe a ser atribuída

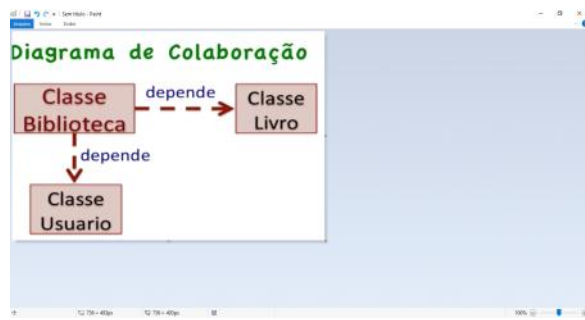
- Se corresponder a responsabilidade óbvia, buscar outra classe
- Se não, busque classe para atribuir

## Passo 6 - Descrever a lógica das responsabilidades / Identificar Colaborações

Lógica "emprestar livro" da Biblioteca:

- ✓ Tem um livro para empréstimo para dado usuário
- ✓ Marca livro como emprestado - classe **Livro** (colaboração)
- ✓ Anexa usuário como "emprestador" do livro - classe **Livro** (colaboração)
- ✓ Anexa livro a lista de livros do usuário - classe **Usuario** (colaboração)

Obs: descrição da lógica é importante para encontrar novas responsabilidades.





# Sobre o módulo

terça-feira, 9 de maio de 2023

20:34

Olá! Bem-vindo à semana 3 do curso Orientação a Objetos com Java! Nesta semana você aprofundará seu contato com Testes de Unidade e Diagramas de Classe da UML, bem como com os conceitos de dependência e contrato de classe. Ao final desta semana, você será capaz de: 1) testar com JUnit o comportamento de classes em Java; 2) projetar e representar classes com diagrama de classes da UML

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/home/week/3>>

# Sobre o módulo

quinta-feira, 11 de maio de 2023 18:10

Olá! Bem-vindo à semana 3 do curso Orientação a Objetos com Java! Nesta semana você aprofundará seu contato com Testes de Unidade e Diagramas de Classe da UML, bem como com os conceitos de dependência e contrato de classe. Ao final desta semana, você será capaz de: 1) testar com JUnit o comportamento de classes em Java; 2) projetar e representar classes com diagrama de classes da UML

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/home/week/3>>

# Testes de Unidade

quinta-feira, 13 de abril de 2023 10:53

Foi feito um exercício onde foi criado uma classe de teste, com 3 testes e os métodos @Before, @BeforeClass, @After, @AfterClass

Projeto **AntesDepois**

# Importância de Testes

quinta-feira, 13 de abril de 2023 10:54

Vídeo explicando sobre teste...

Os métodos ágeis consideram os testes muito importantes e agora fazem parte de todo ciclo de desenvolvimento!

A cobertura de testes é uma métrica da qualidade do código!

Com testes automatizados, é mais seguro realizar modificações no software!

"Criar um software sem testes é como operar um paciente sem monitorar seus sinais vitais!"

"Depois que você adquire a prática de testar o seu software, isso acaba se tornando um hábito que você não larga mais!"

"hoje em dia com todos os recursos e técnica disponíveis, não é mais aceitável um desenvolvedor **profissional** fazer o software sem testes"

# Testes Automatizados com Junit

segunda-feira, 17 de abril de 2023 11:38

Existem vários tipos de teste, que podem ser utilizados pra diferentes propósitos!

## **Unidade**

Teste uma única classe ou um único método.

## **Integração**

Testa a colaboração de um grupo de classes.

## **Funcional**

Teste o software como um todo de acordo com os requisitos.

Esses tipos de teste podem ser criados da mesma forma, e o que varia é o escopo.

## **JUnit**

É praticamente um padrão para a criação de automação de testes em Java.

É muito utilizado para testes de unidade, mas pode ser utilizado para outros tipos também.

- Uma classe de testes do JUnit não precisa de nada especial
- Um método de testes precisa receber a anotação **@Test**
- Não se esqueça de importar as anotações e classes do JUnit
- O método de testes deve exercitar a funcionalidade da classe testada (Rodar a funcionalidade)
- Crie o cenário necessário para que a funcionalidade desejada possa ser exercitada
- O último passo é verificar se o resultado da execução foi o esperado

# Hands-on: Testando com JUnit na Prática

terça-feira, 18 de abril de 2023 11:00

Como exercício foi criado os testes para a classe **Carro** do projeto **Carros**.

Adicionar JUnit no projeto:

Botão direito do mouse em cima do projeto

Properties

Java Build Path

Libraries

Classpath - Add library

JUnit - Next - Finish

Quando criamos teste, por costume é separado o código do teste, do código que estamos desenvolvendo.

No projeto: File -> New -> Source Folder ; nome "teste"

Quanto testes são suficientes para uma determinada classe? Escreva o que acha sobre isso!

Essa resposta não é fácil e nem trivial! Pense que cada cenário que o teste não exercita pode possuir um problema em sua implementação. Se faça a seguinte pergunta: que testes uma outra pessoa precisaria fazer para me convencer que a classe está funcionando corretamente? E tente implementar esses testes!

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/gZZLE/hands-on-testando-com-junit-na-pratica>>

# Antes e Depois de um Teste

quarta-feira, 19 de abril de 2023 10:54

"Uma funcionalidade do JUnit que permite que a gente execute código antes e depois de testes ou uma suíte de testes"

## **@Before**

Utilizado em métodos para executar antes de cada método de teste

## **@After**

Utilizado em métodos para executar depois de cada método de teste

@Before

```
public void before() {...}
```

@After

```
public void after() {...}
```

Exemplo:

Tenho o testA, testB, testC, então ele faria:

@before -> testA -> @after

@before -> testB -> @after

... Executa antes e depois de cada método de teste

E se eu precisar de algo que irá rodar antes de todos os testes?

## **@BeforeClass - @AfterClass**

@BeforeClass

```
public void beforeAll() {...}
```

@AfterClass

```
public void afterAll() {...}
```

Imagine uma classe de testes em que como parte do cenário de cada teste eu preciso ter um arquivo, sempre com o mesmo conteúdo, que é apagado durante a execução. Onde seria o melhor lugar para criação desse arquivo?

R: Em um método @Before

# Hands-on: Métodos Before e After no JUnit

quarta-feira, 19 de abril de 2023 11:18



# Diagramas UML

quarta-feira, 19 de abril de 2023

12:28

# Preciso de Diagramas?

quarta-feira, 19 de abril de 2023 12:28

Por que preciso de um diagrama?

- O diagrama ajuda a comunicar visualmente e eficientemente suas ideias.
- Permite explorar o domínio do problema, pensando como as coisas são relacionadas.
- Ajuda a explorar diferentes soluções alternativas que poderiam ser aplicadas.
- Permite ter uma visão geral do software como um todo, e não apenas de uma pequena parte dele.
- Excelente ferramenta para discutir ideias e o design do software em grupos.

**Cuidado ao seguir cegamente um processo e criar um diagrama sem saber seu propósito!**

"Evite criar diagramas de forma cega, só porque alguém ou algum processo disse que era importante"

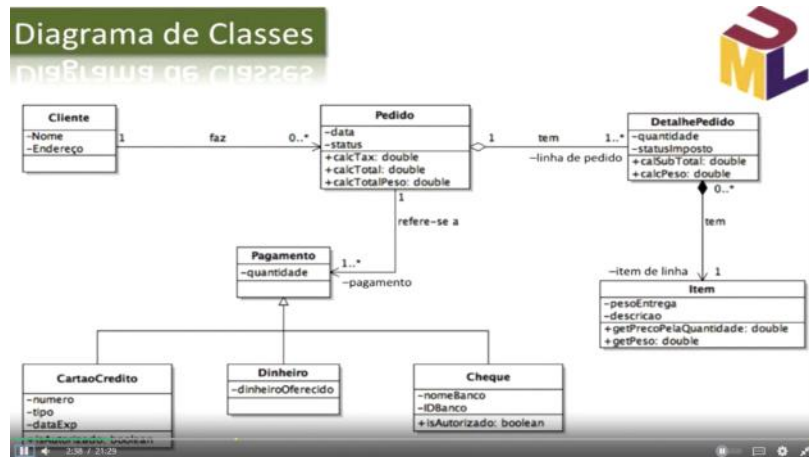
"Sempre entenda o motivo e quem é o público alvo do diagrama"

# Diagrama de Classes UML: Classe, Associação e Multiplicidade

quarta-feira, 3 de maio de 2023 18:27

Diagrama de Classes:

- Visão geral das classes de um sistema, explicitando os relacionamentos de suas classes.
- Unified Modeling Language (UML) [www.uml.org/](http://www.uml.org/)



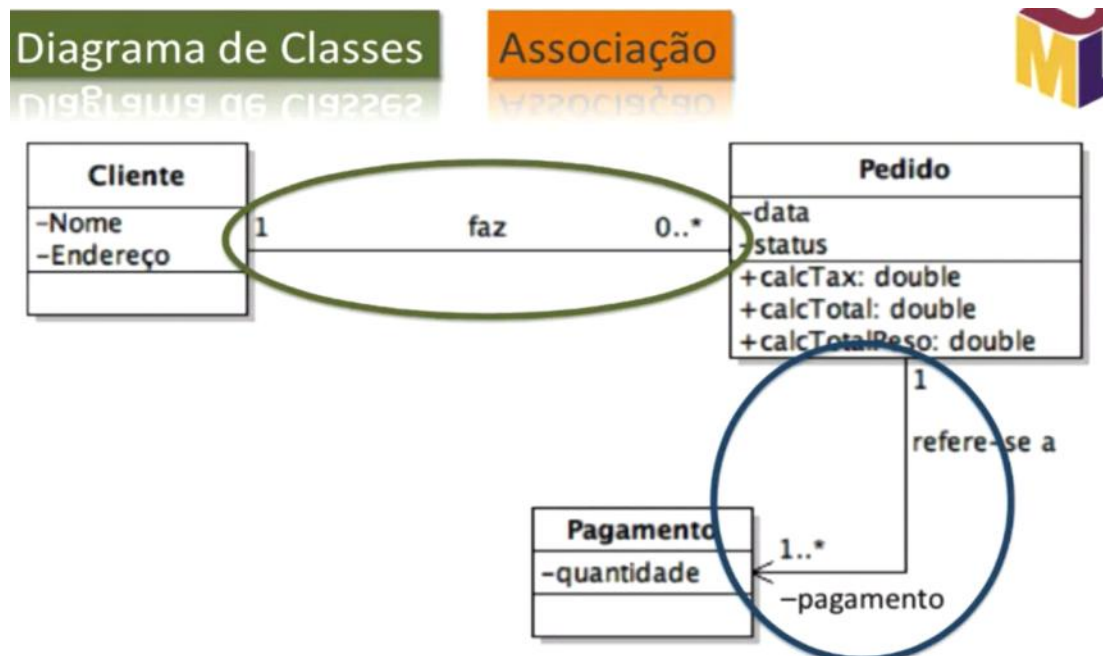
Relacionamentos entre classes:

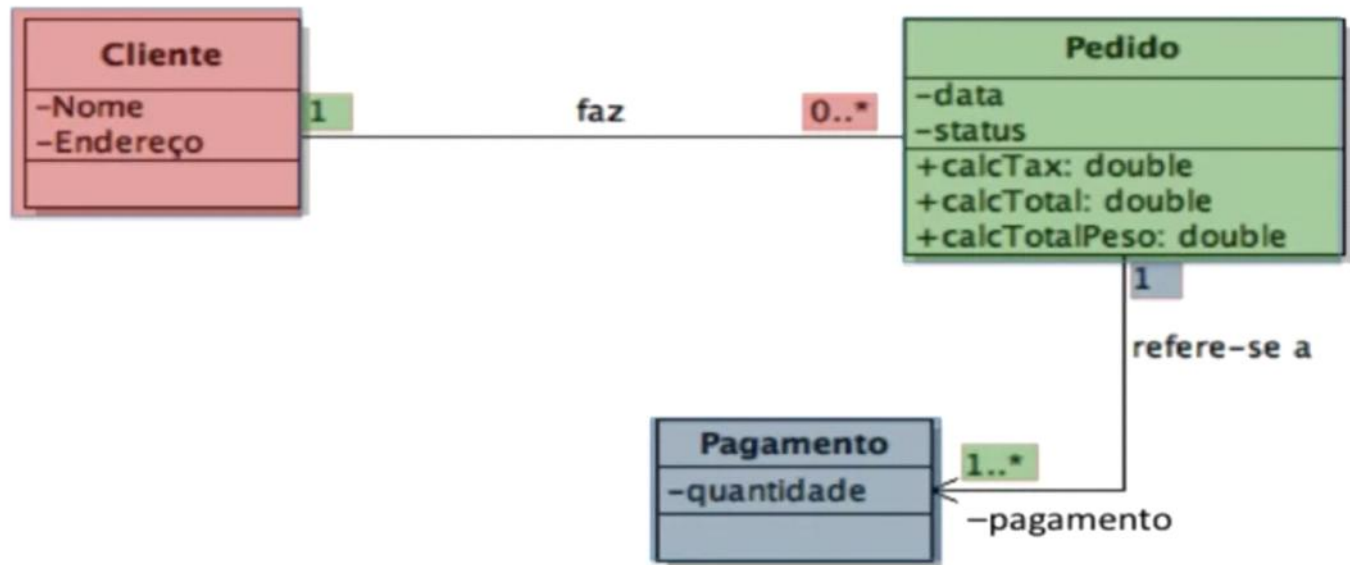
Associação

- Agregação
- Composição

Herança ou generalização:

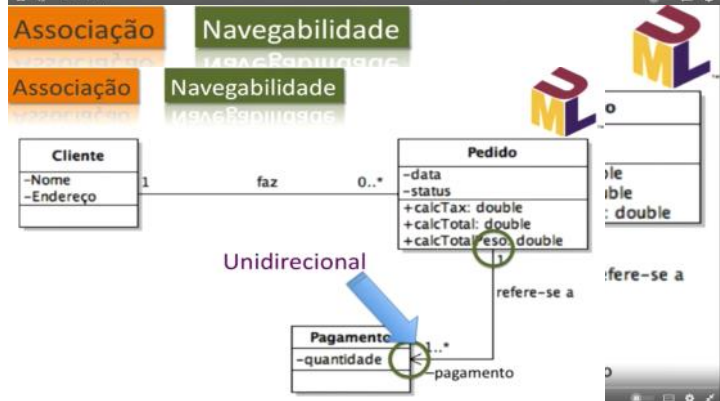
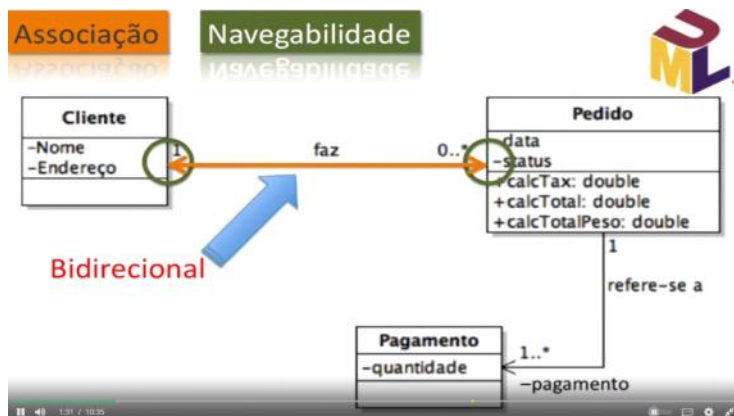
- Dependência





## Diagrama de Classes UML: Navegabilidade, Agregação, Composição e Herança

quinta-feira, 4 de maio de 2023 18:27



Na agregação, se um objeto agregador for eliminado, os objetos agregados serão eliminados também! Verdadeiro ou Falso?!

Falso

Na agregação, se um objeto agregador for eliminado, os objetos agregados continuarão a existir, pois eles já existiam antes de serem agregados, de modo que quando o objeto agregador for removido, isso não afetará os objetos agregados, pois a vida deles não dependia da vida do objeto agregador!

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/VK0X4/diagrama-de-classes-uml-navegabilidade-agregacao-composicao-e-heranca>>

Na composição, se um objeto que compõe outros objetos for eliminado, os objetos da composição serão eliminados também! Verdadeiro ou Falso?!

Verdadeiro

Como os objetos de uma composição são criados dentro do objeto que os compõem, a sua vida está atrelada à vida do objeto que os compõem; de modo que quando ele for eliminado, todos os objetos da composição serão eliminados juntos!

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/VK0X4/diagrama-de-classes-uml-navegabilidade-agregacao-composicao-e-heranca>>



Herança ↓

Generalização ↑ Especialização ↓

Relacionamento é-um ↑



Símbolo de herança:

Triângulo vazio



Classe Abstrata: não posso criar instâncias, somente transmitir métodos comuns em sub classes;  
Classe Concreta: posso criar instâncias;

# Colaboração, Dependência e Contrato de Classes

terça-feira, 9 de maio de 2023

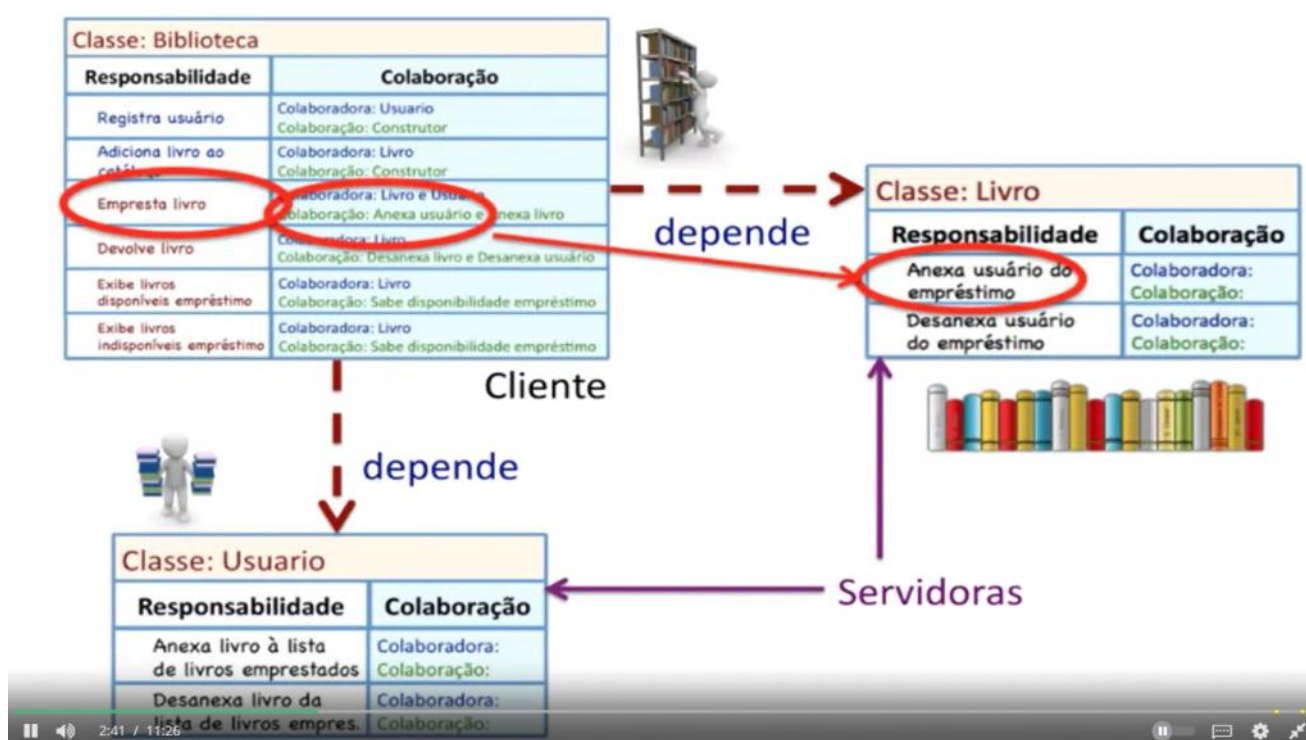
18:24

# Colaboração, Dependência e Classes Cliente e Servidora

terça-feira, 9 de maio de 2023 18:25



Se um objeto Biblioteca não pode existir sem outro objeto Livro, então o objeto Biblioteca **depende** do objeto Livro



Foi mostrado anteriormente que a classe Biblioteca depende das classes Livro e Usuario. Então, podemos denominar a classe Biblioteca de classe servidora e as classe Livro e Usuario de classes clientes! **Verdadeiro ou Falso?!?**

## **FALSO**

A situação é falsa porque uma classe é cliente quando ela necessita ou depende da colaboração de uma classe servidora, que tem a responsabilidade ou método que ela requer. Nesse sentido, a classe Biblioteca depende da colaboração das classes Livro e Usuario, de modo que a classe Biblioteca é que é uma classe cliente e as classes Livro e Usuario é que são classes servidoras!



# Caracterizando Responsabilidade Pública

terça-feira, 9 de maio de 2023

18:44

Estamos considerando 2 tipos de responsabilidades:

- Responsabilidade Pública
- Responsabilidade Privada

Relembrando: responsabilidade é o que uma classe **sabe** ou **faz**.

## Responsabilidade Pública

- É colaboração de pelo menos uma responsabilidade de classe cliente
- UML: prefixo "+"
- Java: **public**



## Caracterizando Responsabilidade Privada e Contrato de Classe

terça-feira, 9 de maio de 2023 18:53

### Responsabilidade Privada

- É parte do funcionamento interno da classe
- UML: prefixo "..."
- Java: `private`



### Responsabilidade Privada

- Não pode ser solicitada por outras instâncias de quaisquer classes
- Inclusive, nem da própria classe da responsabilidade privada



### Responsabilidade Privada

- Não pode colaborar com instância de subclasse
- Ou seja, método privado não é herdado



### Questão

No caso de `metodoPrivado()`, da classe `ConcretosA`, a linha de código `"this.metodoPrivado()"` não pôde ser usada porque, por ser método privado, `metodoPrivado()` não foi herdado pela classe `ConcretosB`. Contudo, para substituí-lo por "super" não funciona: `"super.metodoPrivado()"`

☐ Não

☐ Sim

☒ Sim  
É utilizado por todos métodos públicos da classe `ConcretosA` e também em alguns da classe `ConcretosB` privados, sendo que "this" aponta para "super" e não há funções.

[Ver](#) [Resposta](#)

### Responsabilidade Privada

- É colaboração interna de responsabilidade privada ou pública da instância da própria classe





### Responsabilidade Privada

- Não deve ser mantida na classe se não for colaboradora direta ou indireta de responsabilidade pública



### Contrato de uma Classe

- Contrato é o conjunto de serviços/comportamentos de um objeto
- Pode ser requisitado por (para colaborar com) outros objetos



### Contrato de uma Classe (cont.)

- Contrato também é conhecido por:
  - Conjunto de Responsabilidades Públicas
  - Protocolo
  - Interface Padrão
  - Interface Externa
  - Interface da classe



# Sobre o módulo

terça-feira, 9 de maio de 2023

20:35

Olá! Bem-vindo à semana 4 do curso Orientação a Objetos com Java! Nesta semana você aprofundará seu contato com Herança e Modificadores de Acesso. Ao final desta semana, você será capaz de 1) projetar e estruturar programas Java com base em boas práticas no uso de herança, 2) além de garantir acoplamento baixo entre classes pelo uso adequado de modificadores de acesso

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/home/week/4>>

# Conhecendo a Herança

quinta-feira, 11 de maio de 2023 18:11

# Trabalhando com Níveis de Abstração

quinta-feira, 11 de maio de 2023

18:11

Já entendemos que como as classes são as abstrações a partir das quais são criados os objetos!

**Um gato** - abstrato

**Garfield** - concreto (pois ele já tem suas características)

Podemos ter várias níveis de abstração.

**Felino** - mais genérico

**Gato de rua** - mais específico

Gerente é um Empregado

Gerente extends Empregado

A herança é um recurso da orientação a objetos que permite que classes especializem ou generalizem outras classes.

A partir da herança é possível estender uma classe, adicionando atributos e modificando comportamento.

# Utilizando Herança

quinta-feira, 11 de maio de 2023 18:33

Empregado seria a superclasse

Gerente seria a subclasse

Que outras classes poderiam ser subclasses de Funcionario?

Supervisor e Diretor

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/LP7iS/utilizando-heranca>>

Quando é feito a herança tem coisas que **Pode** e coisas que **Não Pode**

## **Pode:**

- Adicionar métodos
- Adicionar atributos
- Modificar métodos

## **Não Pode:**

- Remover métodos
- Remover atributos
- Estender outra classe (somente 1 classe pode ser estendida)

Ao estender uma classe é preciso **honrar** o contrato da superclasse, a partir de seus métodos e atributos.

É possível remover, na classe, um método da superclasse?

Não

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/LP7iS/utilizando-heranca>>

A herança múltipla existe em algumas linguagens, mas em Java não é permitida.

Em Java, todas as classes estendem a classe **Object**!

Object é a classe que está no topo da hierarquia das classes em Java. (toda classe é um Object).

# Herança: Especialização e Generalização

quinta-feira, 11 de maio de 2023 18:52

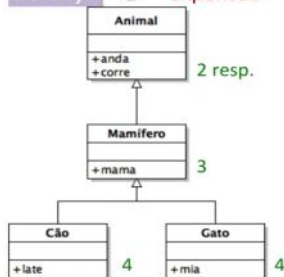
Especialização: Olha da classe para a subclasse

Generalização: Olha da classe para a superclasse

Para exemplificar, pensando na classe Carro:

Ela tem responsabilidades do tipo **Sabe**, que seria a potência do motor e a velocidade  
E responsabilidades do tipo **Faz**, que seria acelerar e frear.

Herança  Expansão



Relacionamento **é um**, usado para confirmar se uma subclasse faz sentido.  
Gato **é um** Mamífero

Generalizar 



1. Responsabilidades comuns

2. Descrições análogas

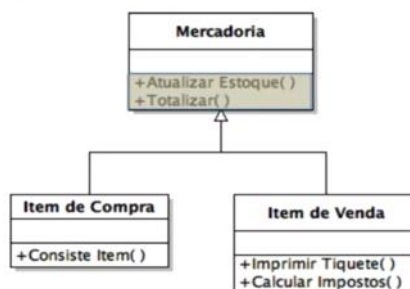
Generalização 

- ◆ Cria superclasse com base em comportamento comum de classes existentes
- ◆ As classes existentes tornam-se subclasses dessa nova classe
- ◆ O comportamento comum é deslocado das subclasses para a superclasse

Condições para Generalizar

1. Responsabilidades comuns
2. Descrições análogas
3. Relacionamento **é-um** respeitado!

Generalização 



3. Relacionamento **é-um**

# Hands-on: Entendendo a Herança

terça-feira, 16 de maio de 2023

19:23

Foi criado o projeto **Banco**. E o prof. deu explicações sobre herança e teste unitário também



# Acesso aos Objetos

quarta-feira, 17 de maio de 2023 18:10

# Modificadores de Acesso

quarta-feira, 17 de maio de 2023 18:10

Os modificadores de acesso podem ser adicionados em membros da classe para definir quem tem acesso a ela.

**private:** só pode ser acessado por membros da própria classe.

**public:** qualquer classe pode acessar os métodos públicos.

**protected:** somente subclasses e classes do mesmo pacote podem acessar.

**default:** acessível para todas as classes do mesmo pacote. (aqui é quando não coloca nada na declaração da variável. Ex: `int valor`)

**Cuidado para não expor demais os membros de sua classe!**

Algumas variáveis só podem ser modificadas de certas formas.

O acesso direto a essa variável pode quebrar o funcionamento da classe.

Uma subclasse também pode modificar uma variável de forma inválida, quebrando o funcionamento da classe.

Todo membro de uma classe que pode ser acessado por outra, se torna mais difícil de ser modificado.

Ex: Imagina que você tem uma **classe** e nesta **classe** tem um **Array**, se de repente eu trocar este **Array** por uma **Lista**, se tiver **outra classe acessando direto o Array**, **não vou ter que mexer só na minha classe, vou ter que alterar a minha e a outra, então quanto mais classes estiverem acessando, pior.**

# Hands-on: Modificadores de Acesso na Prática

quarta-feira, 17 de maio de 2023 18:32

Foi criado o projeto Experiencia acesso.

# Aprofundando na Herança

quarta-feira, 17 de maio de 2023

18:48

# Sobreposição de Métodos

quarta-feira, 17 de maio de 2023 18:48

Uma classe pode subscrever métodos da superclasse.

Porque sobrescrever um método não quebra o contrato da superclasse?

R: Observe que o método disponibilizado pela superclasse poderá ainda ser invocado na subclasse, porém terá um comportamento diferente. O contrato de disponibilizar um método com aquela assinatura foi mantido!

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/eBK29/sobreposicao-de-metodos>>

A subclasse tem que obedecer o contrato.

O **contrato** é o mesmo!

O **comportamento** é diferente!

```
public class Empregado{
    public double liquido(){
        return salario * 0.85;
    }
}

public class Gerente
    extends Empregado{
    public double liquido(){
        return salario * 0.85 + bonus;
    }
}

public class Gerente
    extends Empregado{
    public double liquido(){
        return super.liquido() + bonus;
    }
}
```

*Se mudarem as taxas preciso modificar em dois locais!*

Você pode usar **super** para invocar o método na superclasse!

Uma **classe** com o modificador **final** não pode ser estendida por outras!

Um **método** como o modificador **final** não pode ser sobrescrito!

**final em variáveis tem outro significado!**

```
final Empregado e = new Empregado();
```

**✗** `e = outroEmpregado;`  
*Não pode trocar a referência!*

**✓** `e.setNome("João");`  
*Mas pode modificar o objeto!*

# Classes Abstratas

terça-feira, 23 de maio de 2023 18:53

Pense em um veículo...

Qual?

Existem vários...

Alguns conceitos são muito abstratos para serem instanciados.

Para estes conceitos podem ser criadas **classes abstratas**.

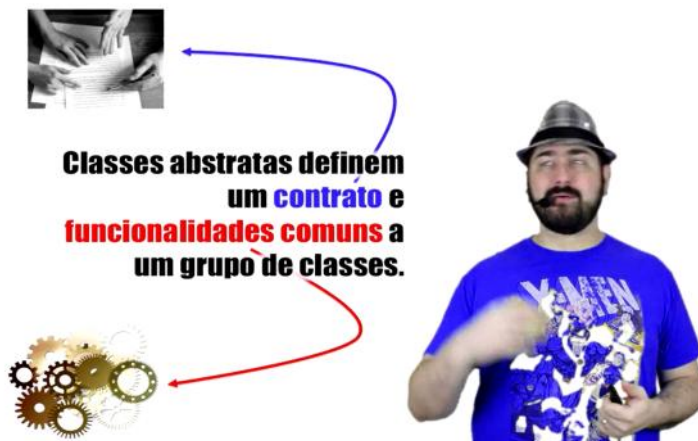
- Pode ser utilizada para ser estendida por outras classes
- Não pode ser instanciada diretamente

O veículo é muito abstrato por exemplo, então não faz sentido `Veiculo v = new Veiculo();`, ele é muito abstrato.

Uma classe abstrata pode possuir métodos abstratos, que precisam ser implementados pelas subclasses.

```
public abstract void mover();
```

Método abstrato: todos sabem fazer aquilo, mas ela não sabe como fazem.



Não faz sentido uma classe abstrata ser instanciada, pq ela pode ter métodos que não tem definição, quem irá definir o método é a subclasse. Classe abstrata é o "contrato". Define a assinatura do método e deixa a implementação para a subclasse.

```
public abstract class Veiculo{  
    public abstract  
        void mover();  
    public abstract  
        String getPosicao();  
}
```

**Uma classe abstrata  
pode possuir métodos  
abstratos, que precisam  
ser implementados  
pelas subclasses.**



# Cadeia de Construtores

quarta-feira, 24 de maio de 2023 18:33

Como inicializar na subclasse o que é da superclasse?

Superclasse

atributoA

atributoB

atributoC

Em Java, um construtor sempre chama outro como a sua primeira ação!

Invocador construtor da superclasse: super()

Invocar construtor da própria classe: this()

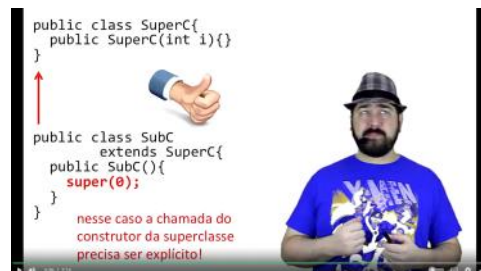


Se você não chama o construtor de forma explícita, ele cria implicitamente.

Por default:

É criado um construtor sem parâmetros se a classe não definir nenhum.

É invocado o construtor sem parâmetros da superclasse se não invocar nenhum.



# Hands-on: Cadeia de Construtores na Prática

quarta-feira, 24 de maio de 2023 18:45

Vamos ver como funciona a cadeia de construtores dentro de uma hierarquia de classe.

Projeto CadeiaConstrutores

```
public static void main(String[] args) {  
    LaDeBaixo obj = new LaDeBaixo();  
}
```

Construtor Pai de Todos

Construtor DoMeio - antes desse rodar chama o de cima

Construtor LaDeBaixo - antes desse rodar chama o de cima

Só é criado o construtor vazio, quando não defino nenhum.

O construtor sempre é a primeira coisa a ser chamada na classe.

Se a classe não tem um construtor sem parâmetros, na classe que estende, tenho que chamar este construtor com parâmetro. Ex: `super("parametro")`



# Hands-on: Herança na Classe Carro – Parte 1

quinta-feira, 25 de maio de 2023

18:31

Refatoração na classe Carro

# Hands-on: Herança na Classe Carro – Parte 2

quinta-feira, 25 de maio de 2023 20:18

CarroDeCorrida, a ideia é que ela seja a superclasse das classes diferentes de carro.

Continuação do projeto Carro

# Hands-on: Herança na Classe Carro – Parte 3

sexta-feira, 26 de maio de 2023 11:08

A classe Corrida vai se basear só na classe abstrata, para ela não interessa qual o algoritmo de acelerar, ela simplesmente vai mandar os carros acelerarem.

Mapa, é uma estrutura de dados que tem uma chave e um valor. Na Corrida, o carro é a chave e a distância que o carro já percorreu, será o valor.

A implementação do método acelerar() que será chamada será: Da classe utilizada para instanciar o objeto passado

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/ea6T5/hands-on-heranca-na-classe-carro-parte-3>>

# Sobre o módulo

sexta-feira, 26 de maio de 2023 19:29

Olá! Bem-vindo à semana 5 do curso Orientação a Objetos com Java! Nesta semana você aprofundará seu contato com os conceitos de Encapsulamento, Acoplamento entre Classes e Interfaces em Java. Ao final desta semana, você será capaz de 1) projetar e estruturar programas Java evitando quebras de encapsulamento e propiciando acoplamento baixo entre classes, 2) além de garantir acoplamento abstrato entre classes pelo uso adequado de interfaces em Java

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/home/week/5>>

# Encapsulamento

sexta-feira, 26 de maio de 2023

19:30

# Importância do Encapsulamento

sexta-feira, 26 de maio de 2023

19:30

"o encapsulamento é considerado um dos pilares da programação orientada a objetos"

Como você trabalha com sistema que tem algumas milhões de linhas de código?  
Você acha que o desenvolvedor desse sistema vai conhecer uma por uma, todas essas linhas de código?

Não. Então o que acontece,  
ele precisa conhecer a parte muito bem a parte que ele está trabalhando,  
e ele precisa conhecer essa superfície que a gente  
falou dos módulos que interagem com aquele que ele está trabalhando.  
Então a ideia é a seguinte, se eu estou fazendo por exemplo,  
sistema que lida com imagens de satélite,  
e eu preciso utilizar por exemplo, filtro de imagem, preciso fazer processamento  
de imagem eu posso ter componente que eu chego para ele e falo assim: olha,  
faz este processamento nessa imagem para mim,  
aí eu não preciso compreender como ele faz esse processamento.  
Eu só tenho que conhecer a superfície, ou seja,  
aquela casca que eu vou interagir no momento de lidar com aquele componente.  
Então quando a gente está desenvolvendo sistema maior é interessante que  
a gente conheça o pedaço do nosso sistema que a gente está trabalhando  
e as interfaces dele com os componentes que interessam para ele.  
E não pense que essa estrutura vai: não estou usando  
orientação a objetos, então vai ficar tudo encapsulado.  
Não senhor.

Isso aí é uma estrutura que você tem que criar no seu software, para que seja  
possível alguém trabalhar numa parte dele sem precisar conhecer todo o resto,  
até mesmo para que quando você trabalhar naquela parte, independente da pessoa  
conhecer o resto ou não o impacto seja naquela parte, e não no resto.  
Sistemas às vezes muito acoplados, você realmente precisa conhecer  
tudo porque uma coisa impacta na outra, uma coisa está misturada com a outra.  
Você vai mexer na lógica, isso está misturado com a interface gráfica,  
às vezes está misturado com o acesso a dados está misturado com o acesso a outros  
sistemas, isso está misturado com a lógica que seria de componente separado com  
cálculo e aí quando você tem que mexer no sistema, você tem que lidar com isso tudo.  
Então, esse encapsulamento é extremamente importante e não vem de graça.

Fique ciente que você tem que pensar o seu software de forma que  
você fale: ok, essa é uma parte independente,  
eu posso criá-la eu posso definir muito bem qual é essa interface para que  
os outros componentes possam interagir com essa parte do meu sistema.

Então isso você tem que projetar, você tentar tudo  
isso que a gente vem falando, separação de responsabilidades,  
que seriam justamente: olha, essa classe é responsável por isso,  
então o que está dentro dela as outras não precisam saber.

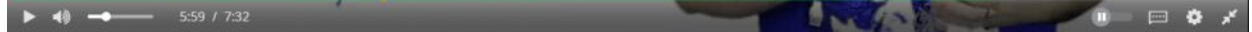
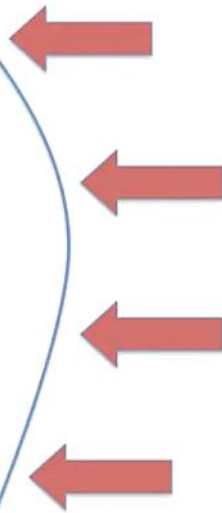
Esta parte maior aqui está dentro desse conjunto de classes,  
e a gente vai fazendo isso com escopos diferentes.

Às vezes uma classe pode encapsular comportamento menor  
componente pode encapsular uma parte maior do meu sistema,  
mas sempre de forma que o resto não precise saber do comportamento interno,  
somente daquela superfície, somente da parte que ele tem que interagir.

Vamos falar mais de encapsulamento, mas espero que nessa aula você tenha  
compreendido como que esse encapsulamento é extremamente importante, principalmente  
se você está desenvolvendo sistemas comerciais, sistema de fácil manutenção  
que uma equipe vai estar trabalhando cima dele e que com certeza você ter  
os componentes encapsulados é extremamente importante para o sucesso do seu projeto.

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/Kuttd/importancia-do-encapsulamento>>

**Por isso é importante pensar em seu software de forma encapsulada dentro de classes e componentes!**



# Métodos de Acesso

sexta-feira, 26 de maio de 2023 19:30

O que você acha de criar métodos get e set para acessar atributos em classes Java?

R:

Nessa aula vamos ver como esses métodos podem nos dar liberdade para poder depois modificar a classe internamente sem quebrar outras classes.

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/8lpvs/metodos-de-acesso>>

Pensando em uma **Atriz**:

Eu tenho muitas informações, mas nem todas eu quero divulgar publicamente! Ou pode ser que publicamente eu queira divulgar de uma forma diferente...

```
public class Atriz {  
    private String nome;  
    private int idade;  
    private String segredo;  
    private Ator amante;  
}
```

**Minhas informações são privadas e eu posso decidir como os métodos vão expor essa informação!**

Os métodos de acesso são uma forma de dar acesso público as informações da classe! (get e set)  
O padrão Java Beans define uma forma padronizada de criar esses métodos de acesso!

"ah mas não faz sentido eu modificar o nome de uma atriz, por exemplo".  
Para uma propriedade **somente-leitura**, basta não adicionar o método "set".

```
public class Atriz {  
    private int idade;  
  
    public void setIdade (int i) {  
        if (i > 0) {  
            idade = i;  
        }  
    }  
}
```

**Um método "set" pode possuir uma lógica para aceitar apenas valores válidos.**

**Em um método "get" o valor retornado não precisa ser exatamente o que está na variável...**

```
public class Atriz {  
    private int idade;  
  
    public int getIdade() {  
        if (idade > 30) {  
            return idade * 0.85;  
        }  
  
        return idade;  
    }  
}
```

"Em um sistema real não é tão absurdo você querer ajustar a informação antes de retornar"



```
public class Atriz {  
    private Date nasc;  
  
    public int getIdade() {  
        long hoje = System.currentTimeMillis();  
        long dif = hoje - nasc.getTime();  
  
        return (int) dif / 1000 / 60 / 60 / 24 / 365.25;  
    }  
}
```

**O formato de armazenamento e retorno não precisa ser o mesmo!**

**Os métodos de acesso permitem uma separação da estrutura interna do contrato externo da classe!**

"Este é um dos reais benefícios do encapsulamento. Da a liberdade de você trabalhar sua classe internamente e manter sua estrutura interna"

**Métodos de acesso vão um pouco além de get e set em todos os atributos da classe.**

# Hands-on: Exemplo de Violação Encapsulamento

sexta-feira, 26 de maio de 2023

19:30

"A ideia é mostrar uma classe Pilha, que é uma estrutura de dados que todos já conhecem. E mostrar que se a gente quebrar o encapsulamento dela, exportar demais as informações, pode causar um funcionamento não desejado"

Projeto PilhaEncapsulada

"Tem gente que acha que encapsulamento é simplesmente colocar os atributos privados. Mas não adianta nada, se você exportar as informações de outra forma".

Foi removido o método setTopo da classe, pois não poderia criar a pilha, incluir e depois mudar o tamanho dela.

# Encapsulamento de Objetos e Arrays

sexta-feira, 26 de maio de 2023

19:30

Se o valor de um atributo do tipo Object ou Array é retornado por um método, o que é retornado?

R: Uma referência ao objeto ou array que está no atributo

De <https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/Wmf8g/encapsulamento-de-objetos-e-arrays>

```
String[] p = atriz.getPremios();  
p[0] = "Framboesa de Ouro";
```

**Modificar o array adquirido faz com que ele seja modificado dentro da classe também!**

```
Ator a = atriz.getAmante();  
a.setName("Fulano");
```

**Objetos modificados também são alterados dentro da classe!**

**Estas duas situações são Quebra de Encapsulamento!**

**Solução 1** - Retorne um clone (copiar o objeto que esta dentro da classe e retornar, ai o que for alterado não esta no objeto).

```
public class Atriz {  
    private Ator amante;  
  
    public Ator getAmante() {  
        Ator at = new Ator();  
        //copia informações amante  
        return at;  
    }  
}
```

**Dessa forma, o objeto que compõe a classe Atriz não será modificado!**

```
Ator a = new atriz.getAmante();  
a.setName("Fulano");
```

```
public class Atriz {  
    private String[] premios;  
  
    public String[] getPremios() {  
        String[] copia = Arrays.copyOf(premios, premios.length);  
        return copia;  
    }  
}
```

**A mesma ideia pode ser utilizada para retornar uma cópia dos arrays!**

```
String[] p = atriz.getPremios();  
p[0] = "Framboesa de Ouro";
```

**Solução 2** - Esconda o Objeto (não retornar ele de forma nenhuma).

```
public class Atriz {
```

```
private Ator amante;

public String getNomeAmante() {
    return amante.getNome();
}

public int getIdadeAmante() {
    return amante.getIdade();
}
}
```

**O objeto da classe Ator não é retornado, e a classe controla totalmente o acesso a ele!**

```
public class Atriz {
    private String[] premios;

    public String getPremio(int i) {
        return premios[i];
    }
}
```

**No caso de arrays, podem ser utilizados métodos que recebem o índice a ser retornado!**

# Acoplamento entre Classes

sexta-feira, 26 de maio de 2023

19:31

# Caracterizando Acoplamento e Duas Situações de Acoplamento Alto

sexta-feira, 26 de maio de 2023 19:31

**Alto acoplamento não é bom!**

## **Acoplamento:**

Descreve os relacionamentos e dependências entre classes.

Acoplamento Alto

**Pior tipo:**

**Internal Data Coupling**

**Acoplamento com Dados Internos da Classe**

Isso deixa os dados públicos

**Getters/Setters**

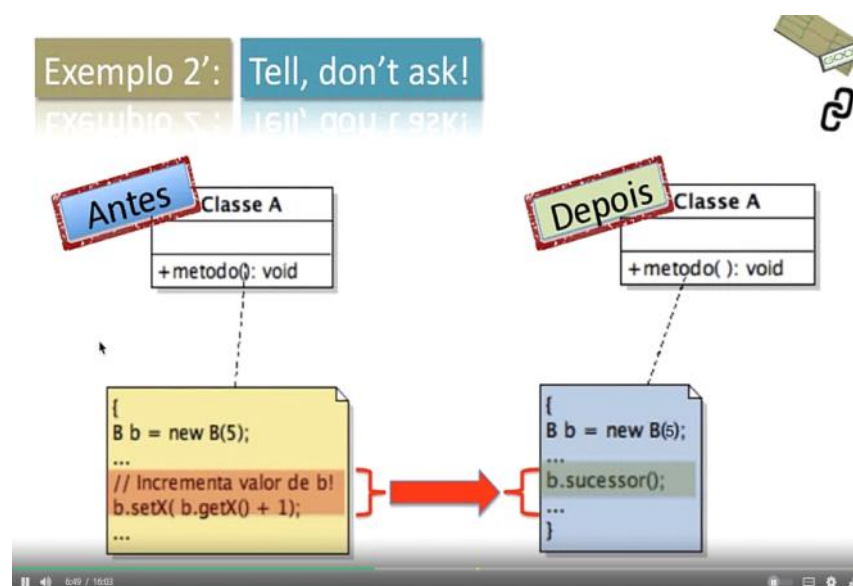
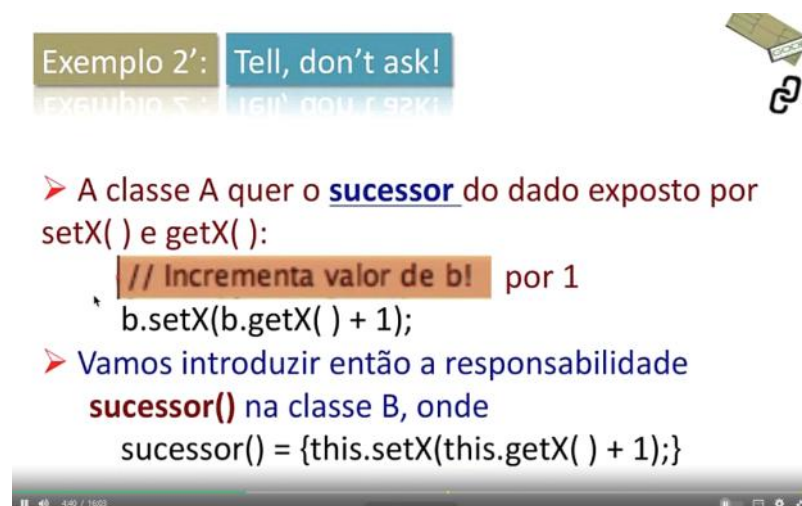
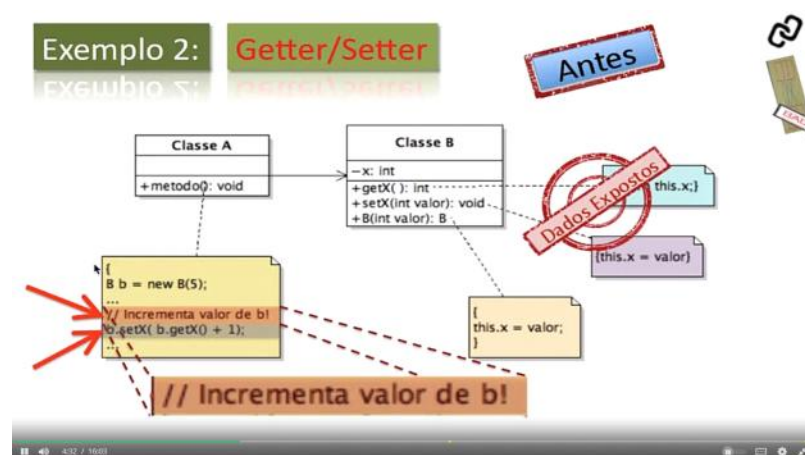
Dados expostos

# Aplicando Princípio "Tell, Don't Ask!" para Obter Acoplamento Baixo

sexta-feira, 26 de maio de 2023 19:31

Ordene, não peça!

- Não peça ao objeto pela **informação** que você necessita para **fazer algum trabalho**.
- Ao invés, peça ao **objeto** que tem a **informação** para **fazer o trabalho** para você



- Lógica distribuída entre as classes de uma aplicação

- Responsabilidade atribuída a classe mais apropriada, de acordo com a sua finalidade.

Seguindo isto, a consequência seria:

- Diminui o acoplamento entre classes devido ao acesso ou a exposição de dados internos
- Diminui necessidade de getters/setters públicos

getters/setters protegido:

Permite acesso a variáveis de instância (por convenção privadas) das superclasse por subclasse!



# Teste para praticar

sexta-feira, 26 de maio de 2023

19:31

**1.**

Pergunta 1

Acoplamento significa a ocorrência de relacionamentos (herança, associação, agregação e composição) e dependências entre classes.

**1 / 1 ponto**

Falso

Verdadeiro

**Correto**

**2.**

Pergunta 2

Acarreta quebra de encapsulamento:

**1 / 1 ponto**

Uso inadequado de getters públicos (métodos de acesso do tipo get)

**Correto**

Acoplamento com dados internos da classe

**Correto**

Uso inadequado de setters públicos (métodos de acesso do tipo set)

**Correto**

**3.**

Pergunta 3

O que você entende pelo princípio "Tell Don't Ask!"?

**1 / 1 ponto**

Pelo o que eu entendi, o princípio significa não deixar o objeto cliente acessar dados do objeto servidor.

**Correto**

Para mim significa que a classe servidora não vai permitir o acesso a dados privados por meio de getters e setters públicos.

**Correto**

Já eu penso que o princípio incentiva o desenvolvedor a construir classe servidora que ofereça serviços que impeçam a vontade de classes clientes acessarem dados internos das classes servidoras, quer por estes dados estarem públicos, quer por getters e setters públicos, quebrando assim o encapsulamento de objetos da classe servidora.

**Correto**

**4.**

Pergunta 4

O que significa em Java a ação expressa pelo comentário do código, lembrando que B é uma classe empregada na exemplificação da videoaula "Aplicando Princípio 'Tell, Don't Ask!' para Obter Acoplamento Baixo":

1  
2  
3  
4  
5  
6

```

. . .
B b = new B(10);
. . .
// incrementar valor de b por 1!
b.setX(b.getX( ) + 1);
. . .

```

**1 / 1 ponto**

"incrementar valor de b por 1" significa aplicar a função sucessor( ) a b; ou seja, faça b = b + 1!

**Correto**

"incrementar valor de b por 1" significa aplicar a função predecessor() a b; ou seja, faça b = b – 1!

**5.**

Pergunta 5

Os métodos de acesso getX( ) e setX(int) da classe B, que antes da aplicação do princípio "Tell, Don't Ask!" eram públicos, passaram a ser protegidos, como se pode ver no trecho abaixo, lembrando que B é uma classe empregada na exemplificação da videoaula "Aplicando Princípio 'Tell, Don't Ask!' para Obter Acoplamento Baixo":

```

public class B {
    . . .
    public void sucessor(int x){
        . . .
    }
    protected int getX( ){
        return x;
    }
    protected void setX(int x){
        this.x = x;
    }
    . . .
    private int x;
}

```

O motivo deles passarem a ser protegidos foi o seguinte:

**1 / 1 ponto**

Garantir o acesso a elas por objetos de subclasses de B

**Correto**

Não expor externamente a variável de instância x de B

**Correto**

Em algumas poucas situações há a necessidade do getter ser público, mas com uso externo controlado, como se fôsse um padrão! Em material de leitura complementar isso será tratado!

Impedir o acesso a elas por objetos de subclasses de B

1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
11  
12  
13  
14

Expor externamente a variável de instância x de B

## 6.

Pergunta 6

Antes de aplicar o princípio "Tell, Don't Ask!", o código que permitia o acesso aos dados internos de objetos B, quebrando o encapsulamento, era o seguinte (lembrando que B é uma classe empregada na exemplificação da videoaula "Aplicando Princípio 'Tell, Don't Ask!' para Obter Acoplamento Baixo"):

```
. . .  
B b = new B(10);
```

```
. . .  
// incrementar valor de b por 1!  
b.setX(b.getX( ) + 1);
```

. . .  
Depois, a linha abaixo do comentário ficou do seguinte jeito:  
**1 / 1 ponto**

```
this.sucessor( );
```

```
b.sucessor( );
```

```
this.getX( ).sucessor( );
```

```
b.getX( ).sucessor( );
```

**Correto**

## 7.

Pergunta 7

Após aplicar o princípio "Tell, Don't Ask!", a classe B recebeu o método público que correspondia ao desejo de quem expunha os dados da B antiga: sucessor( ), como se pode ver no trecho abaixo da classe B, em que não se expôs a lógica dele (lembrando que B é uma classe empregada na exemplificação da videoaula "Aplicando Princípio 'Tell, Don't Ask!' para Obter Acoplamento Baixo"):

```
public class B {  
    . . .  
    public void sucessor( ){  
        ? ? ?  
    }  
    . . .  
    private int x;  
}
```

Como ficou a lógica do método sucessor( )?

**1 / 1 ponto**

```
this.setX(this.getValor() + 1);
```

1  
2  
3  
4  
5  
6

1  
2  
3  
4  
5  
6  
7  
8

```
b.setX(b.getValor() + 1);
```

**Correto**

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/quiz/7CIRg/sobre-acoplamento-entre-classes/view-attempt>>

# Interfaces em Java

sexta-feira, 26 de maio de 2023 19:31

# Interfaces na Orientação a Objetos

quinta-feira, 1 de junho de 2023 13:42

Pense que eu tenho as classes **Cavalo** e **Carro**, **ambos se movem**.  
Como modelar essa característica em comum?

Pensando em **herança**.. **Um cavalo não é um carro**.

Uma relação de herança não faz sentido entre eles! Não faz sentido por exemplo, eu pegar a classe Cavalo e estender de Carro, pra aproveitar a funcionalidade de movimento dela, ou vice-versa. **Não é esse que é o reuso da orientação objeto. A gente só deve usar a herança quando a subclasse, ela é um da superclasse, e nesse caso não se aplica.**

Se tentarmos pensar em uma superclasse comum, quem seria?

O problema é que tanto Cavalo(**Animal**) quanto Carro(**Veiculo**), eles já tem uma hierarquia.

**O que essas classes compartilham é um comportamento, mas elas não tem um conceito em comum!**

O relacionamento da herança, ele tem haver com o que a classe é. Quando a gente fala que é uma **classe A** estende uma **classe B**, é porque a classe A **é um** da classe B.

## Interface

As interfaces podem ser utilizadas quando queremos **abstrair** um comportamento!

A interface também é um tipo de contrato, que define os métodos que a classe precisa ter!

Interface MoveI \*Cavalo implementa MoveI\*

Dizemos que uma classe **implementa** ou realiza uma **interface**!

Classe Corrida **recebe uma lista de objetos que implementam** a Interface MoveI

Qualquer outra classe que implementar a Interface MoveI, poderá ser utilizada na Corrida!

# Interfaces em Java

quinta-feira, 1 de junho de 2023 13:42

## Interface MoveI

```
public interface MoveI {  
    public void mover();  
    public void parar();  
    public double getVelocidade();  
}
```

**Define os métodos que uma classe que a implementa precisa ter!**

Se os métodos de uma interface não possuem implementação, então eles são:

R: abstratos

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/6L3Za/interfaces-em-java>>

**Todo método de interface é abstrato!**

Por esse motivo o uso da palavra "abstract" é opcional!

O que você pode colocar em uma interface?

- Métodos abstratos
- Atributos finais e estáticos

O que não pode?

- Todo o resto

```
public class Cavalo implements MoveI {  
    // precisa implementar todos os métodos da interface  
}
```

```
public abstract class Veiculos implements MoveI {  
    // não precisa implementar todos os métodos da interface,  
    // mas subclasses concretas sim!  
}
```

```
public interface Televisor extends Ligavel, ComCanais {  
    // pode definir mais métodos  
}
```

Uma interface pode estender uma ou mais interfaces!

Isso significa que a interface que estendeu as duas, é como se ela definisse todos os métodos que as outras possuem.

```
public class SmartTV implements Televisor, PlayerVideo {  
    // deve implementar o contrato de todas as suas interfaces  
}
```

Uma classe pode implementar uma ou mais interfaces!

# Exemplo de Interface

quinta-feira, 1 de junho de 2023 13:42

Imagine a implementação de um algoritmo para ordenar listas de objetos!

`Collections.sort(list);`

Mas existem diferentes coisas que podem ser ordenadas! Texto, Números, Datas...

**Inclusive objetos da sua própria aplicação! Que tem uma lógica bem específica do seu negócio.**

Candidatos de um concurso, Investimentos pelo risco, Jogadores pela pontuação...

Poder ser comparado com outros objetos, ou seja, ser "comparável", é um comportamento?

Sim

De <https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/Z5GIl/exemplo-de-interface>

O reuso na **OO** está ligado ao aproveitamento de um código que não foi feito especificamente para a sua classe!

São códigos feitos para um contrato!





# Hands-on: Usando a Interface Comparable

sexta-feira, 26 de maio de 2023 19:32

Vamos ver como eu crio uma classe e faço ela ser ordenada por um método que já existe.

Projeto Concurso, onde será ordenado os candidatos.  
Um candidato pode ser comparado a outro candidato.

# Material Complementar

quinta-feira, 1 de junho de 2023

15:35

# Hands-on: Problema no Encapsulamento de Arrays

quinta-feira, 1 de junho de 2023 15:35

Foi exemplificado no projeto PilhaEncapsulada

Quebra de encapsulamento:

```
public static void main(String[] args) {  
    Pilha p = new Pilha(10);  
    p.empilhar("Eduardo");  
    p.empilhar("Maria");  
    p.empilhar("José");  
    System.out.println(p.topo());  
    System.out.println(p.tamanho());  
  
    Object arrayElementos[] = p.getElementos();  
    arrayElementos[1] = "OUTRO";  
  
    // Desempilhando  
    System.out.println("-----");  
    System.out.println(p.desempilhar());  
    System.out.println(p.topo());  
    System.out.println(p.tamanho());  
}
```

Quais são as alternativas nesse caso para não quebrar o encapsulamento?

Retornar uma cópia do array

**Correto**

Funciona pois uma alteração na cópia não irá afetar o array que está no atributo.

Não retornar o array e encapsular o acesso a seus elementos

**Correto**

Funciona pois nesse caso quem acessa a classe pode ter acesso as mesmas informações, mas não tem acesso direto ao array.

Atenção!

Se o objeto do array for mutável, deve-se tomar o mesmo tipo de cuidado.

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/iYfq8/hands-on-problema-no-encapsulamento-de-arrays>>

# Sobre o módulo

quinta-feira, 1 de junho de 2023 16:05

Olá! Bem-vindo à semana 6 do curso Orientação a Objetos com Java! Nesta semana você aprofundará seu contato com o conceito de Polimorfismo, princípio "Law of Demeter" e Exceções em Java. Ao final desta semana, você será capaz de 1) projetar e estruturar programas Java mais flexíveis e com acoplamento baixo, 2) além de garantir o tratamento adequado de exceções em Java

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/home/week/6>>

# Sobre Polimorfismo

quinta-feira, 1 de junho de 2023 16:13

# Entendendo Polimorfismo

quinta-feira, 1 de junho de 2023 16:13

Na natureza, um animal pode assumir diferentes formas de acordo com a necessidade do estágio de sua vida. (Ex: lagartas de borboleta).

Em um software orientado a objetos, uma classe pode assumir a forma de sua superclasse ou de uma de suas interfaces!

O polimorfismo pode ser utilizado:

Para que uma classe possa ser atribuída a uma variável que espera receber o tipo de sua superclasse

**Correto**

Uma classe pode assumir a forma de sua superclasse

Para que uma classe possa ser atribuída a uma variável que espera receber o tipo de uma de suas interfaces

**Correto**

Uma classe pode assumir a forma de qualquer uma de suas interfaces

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/oFRmr/entendendo-polimorfismo>>

```
Cavalo c = new Cavalo();
```

```
Animal a = c;
```

**Um objeto do tipo Cavalo pode ser atribuído para uma variável do tipo Animal, sua superclasse!**  
(Animal é a superclasse de cavalo).

```
Cavalo c = new Cavalo();
```

```
Movel m = c;
```

**Um objeto do tipo Cavalo pode ser atribuído para uma variável do tipo Movel, uma de suas interfaces!**

String classificar(Animal a) - neste caso Animal pode ser um cavalo, ou um pássaro, ou um escorpião..

**Um objeto pode ser sempre passado em parâmetros do tipo de sua superclasse!**

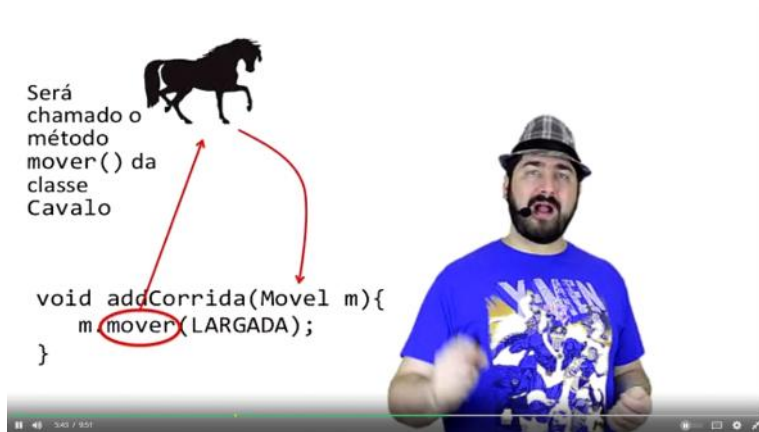
Por exemplo, a classe BemTeVi que estende Passaro que estende Animal.

Void addCorrida(Movel m) - Movel pode ser um cavalo, ou uma bicicleta, ou um carro...

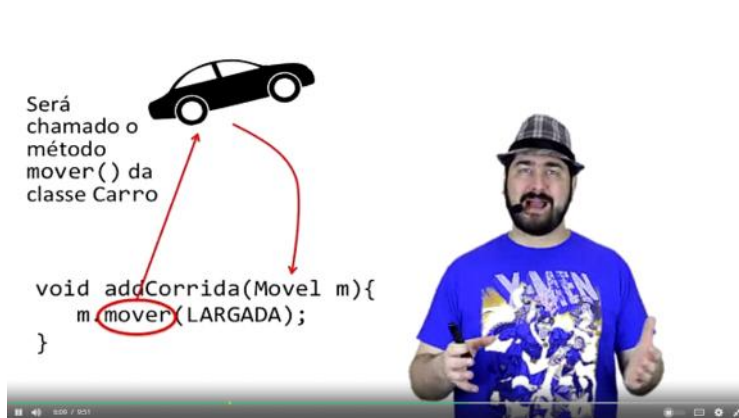
**Um objeto pode ser sempre passado em parâmetros do tipo de umas de suas interfaces!**

**Esses métodos só podem usar os métodos definidos no contrato da classe ou interface!**

Por exemplo, se no **método classificar**, utilizar um **escorpião**, ele **não pode usar o método ferroar**, porque é só **escorpião** que tem, ele vai usar os métodos da classe Animal que é comum a todos.



O método está definido na Interface mas está implementado na classe Cavalo.



O método invocado é definido no **Contrato**(Interface ou Superclasse) mas sua implementação é definida pela **Classe**.

O comportamento do método é de acordo com a classe que estou passando pra ele, o objeto que estou passando pra ele.

**O mesmo código pode ter diferentes comportamentos dependendo do objeto que está utilizando!**

É possível estender o comportamento de uma classe ou método passando uma nova implementação da abstração que ela recebe.



# Hands-on: Interfaces e Polimorfismo

quinta-feira, 1 de junho de 2023 16:13

## Projeto Interfaces

Qual o nome do conceito que nos permite atribuir as instâncias de classes (no caso Cachorro, Carro e Bateria) que implementam uma interface no array do tipo dessa interface (no caso Barulhento)?

Polimorfismo

**Correto**

Guarde bem essa palavra que iremos utilizar muito ela nos próximos cursos!

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/VFeN4/hands-on-interfaces-e-polimorfismo>>



# Law of Demeter

quinta-feira, 1 de junho de 2023

16:15

# Identificando dependências mais Complexas

quinta-feira, 1 de junho de 2023 16:15

Aplicamos anteriormente o princípio "Tell, don't ask" para resolver o acoplamento alto devido a uso de getters/setters de uma classe!

A ideia é aplicar uma combinação do princípio "Tell, don't ask" com "Law of Demeter".

Até agora, o getter usado devolvia apenas um valor de dados simples, como int e double, por exemplo!

O que aconteceria se o getter devolvesse um **objeto de classe** da aplicação?

No trecho de código "item.getProduto().getPeso()", qual parte corresponde a um objeto anônimo da classe Produto? Indicamos isso por um par de parêntesis em negrito e objeto anônimo corresponde a objeto que não é referenciado por variável de instância, nem por variável temporária!

item.getProduto( ).**(getPeso( ))**

**(item.getProduto( ))**.getPeso( )

item.**(getProduto( ))**.getPeso( )

**(item)**.getProduto( ).getPeso( )

**Correto**

**(item.getProduto( ))**: getProduto( ) é uma mensagem enviada ao objeto item, da classe ItemPedido, cuja ação é devolver um objeto da classe Produto, que no caso corresponde a um objeto anônimo da classe Produto porque não é referenciado por variável de instância nem por temporária! Envia-se, em seguida, a mensagem getPeso( ) a esse objeto anônimo da classe Produto, que devolve o peso do produto representado.

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/MOAlX/identificando-dependencias-mais-complexas>>

```
public class Frete {  
  
    public Frete(PedidoVenda pedido) {  
        this.pedido = pedido;  
    }  
  
    public BigDecimal getPesoPedido() {  
        // Obter peso total do pedido  
        BigDecimal pesoTotal = new BigDecimal(0D);  
        for (ItemPedido item : pedido.getItems()) {  
            pesoTotal =  
                pesoTotal.add(item.getProduto().getPeso().multiply(item.getQuantidade()));  
        }  
        return pesoTotal;  
    }  
  
    private PedidoVendas pedido;  
}
```

**Dados expostos!**



# Aplicando o princípio Law of Demeter

quinta-feira, 1 de junho de 2023 16:15

Uma solução é aplicar uma combinação dos princípios "Tell, don't ask!", Redirecionamento/Delegação e "Law of Demeter" nessa situação!



**LoD** Um método *m* de um objeto deve invocar apenas os métodos dos seguintes tipos de objetos:

1. O próprio objeto (*this*)
2. Os parâmetros de *m*
3. Quaisquer objetos que *m* instancia

Isso ajuda a desenvolver softwares com menos dependência.

**LoD** Um método *m* de um objeto deve invocar apenas os métodos dos seguintes tipos de objetos:

1. O próprio objeto (*this*)
2. Os parâmetros de *m*
3. Quaisquer objetos que *m* instancia
4. Objetos componentes diretos do objeto (variáveis de instância de relacionamento)
5. ~~Objetos globais~~

Redirecionamento é quando um método *m* apenas repassa sua responsabilidade para outro método de outra classe, cujo nome desse outro método é igual a *m*!

**Correto**

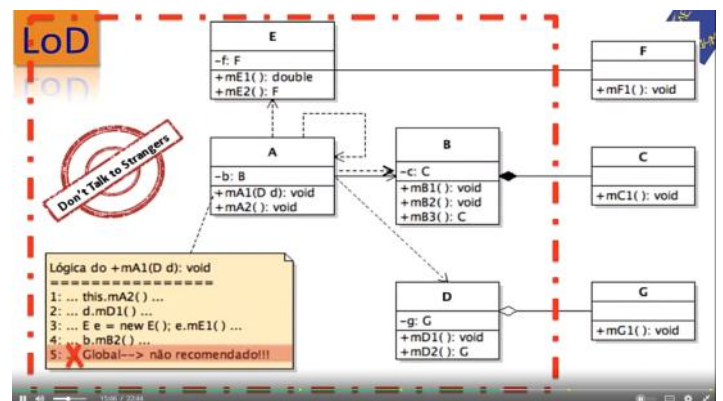
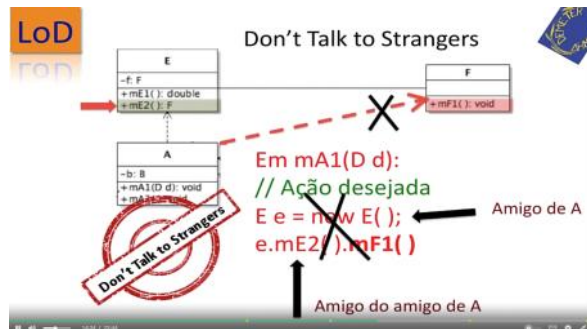
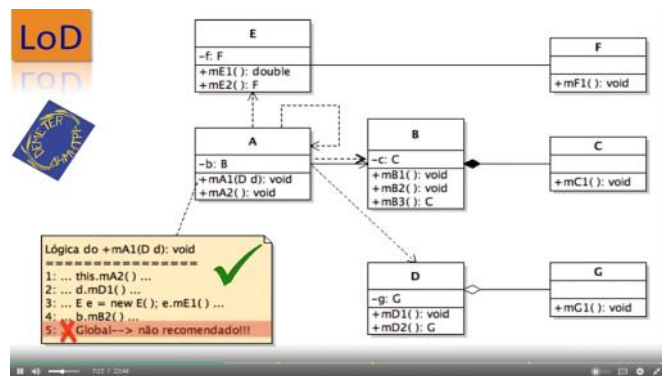
Alguns autores consideram essa definição como estando incluída na definição de delegação, não reconhecendo o termo redirecionamento exclusivo para esse caso!

Delegação é quando um método *m* apenas repassa sua responsabilidade para outro método de outra classe, cujo nome desse outro método é diferente de *m*!

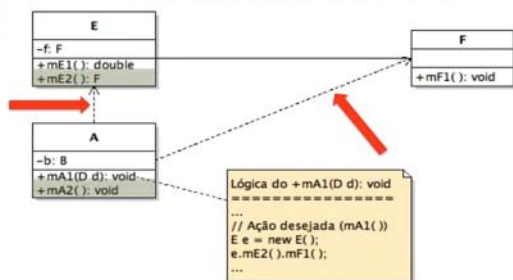
**Correto**

Alguns autores consideram que a definição de delegação também inclui o caso do nome do outro método ser igual a *m*! Ou seja, inclui o que estamos chamando de redirecionamento!

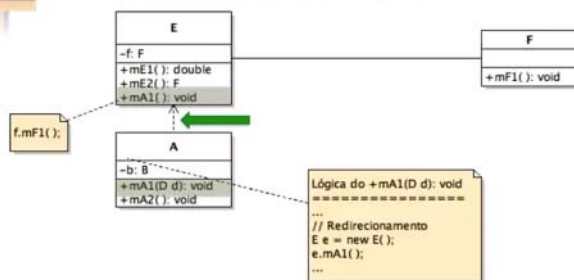
De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/5s87W/aplicando-o-principio-law-of-demeter>>



## Antes: 2 dependências



## Depois: Apenas 1 Dependência



# Exceções em Java

quinta-feira, 1 de junho de 2023 16:16

# Tratamento de Erros

quinta-feira, 1 de junho de 2023 16:16

## Como avisar quem invocou um método que um erro ocorreu em sua exceção?

Uma opção é retornar um valor específico que pode ser verificado em caso de erro. Exemplo abaixo:

```
public double porcao(double valor, double porCento) {  
    if (porCento > 100 || porCento < 0) {  
        return -1;  
    }  
    return valor * porCento / 100;  
}
```

## Isso não é uma boa ideia!

Alguns motivos:

caso alguém utilize desta forma, não irá entender o porque do erro.  
porcao(5000, 120); // por que estou recebendo -1?

-1 pode ser um valor válido  
porcao(-100, 1);

## E se vários erros diferentes forem possíveis?

Abrir arquivo -> bloqueado -> inexistente -> acesso negado

## Nem pense em armazenar erros em variáveis globais!

Se não limpar, pode deixar erros de execuções anteriores!  
Gera problemas com execução concorrente!

**Java utiliza um mecanismo de exceções para o tratamento de erros!**

# Exceções em Java

quinta-feira, 1 de junho de 2023 16:16

**Exceções são classes especiais que representam erros e podem ser lançadas para quem invocou um método.**

```
public class MinhaException extends Exception {  
    // ...  
}
```

Para ser uma exceção, a classe precisa ser subclasse de **Exception**.

```
public void metodo(int param) throws MinhaException {  
    if (error) {  
        throw new MinhaException("erro");  
    }  
}
```

- um método deve declarar explicitamente as exceções que pode lançar
- uma nova instância da exceção deve ser criada para ser lançada

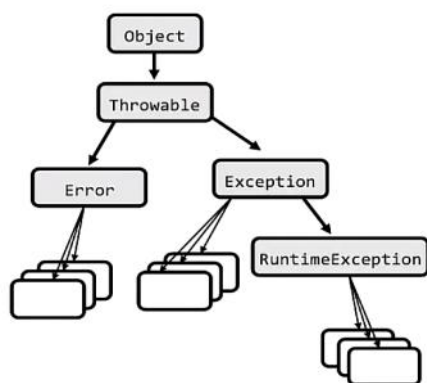
```
public void quemChama() {  
    try {  
        instancia.metodo()  
    } catch (MinhaException e) {  
        // trata erro  
    }  
}
```

Quem chama esse método pode pegar a exceção usando um bloco try/catch.

```
public void quemChama() throws MinhaException {  
    instancia.metodo();  
}
```

Ou o método pode também declarar que joga aquela exceção e deixar quem o chamou tratar.

**Você pode tratar o erro ou passar para o próximo tratar... mas você não pode simplesmente ignorar!**





**Error:**

Indica erros ou condições anormais que normalmente não tem muito a ser feito quando são lançados.

CoderMalfunctionError

FactoryConfigurationError

VirtualMachineError

**RuntimeException:**

Indica problemas normais que não são obrigatórios de serem tratados.

ArithmeticException

ClassCastException

NullPointerException

IndexOutOfBoundsException

**Exception:**

Indica problemas que podem acontecer durante a execução do programa que precisam ser tratados.

SQLException

IOException

PrintException

DataFormatException

O bloco try/catch está sendo utilizado para forçar a saída do bloco while, pois quando chegar ao final do array uma exceção será lançada. Você acha que deve-se utilizar o tratamento de exceções para o controle de fluxo dessa forma?

Não

**Correto**

Blocos try/catch devem ser utilizados para o tratamento de erros. Deve-se evitar forçar o acontecimento de erros, como no exemplo, para que o try/catch seja utilizado para controle de fluxo.

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/wGQfC/excecoes-em-java>>

# Hands-on: Exceções na Prática

quinta-feira, 1 de junho de 2023 16:16

Foi criado o projeto Excecoes

Quais as alternativas para um método que pode lançar uma exceção em sua execução?

Declarar que também lança aquela exceção com a cláusula throws

**Correto**

Nesse caso, o erro será propagado para o próximo método.

Adicionar um bloco try/catch para tratar a exceção

**Correto**

É uma forma de se tratar o erro e não deixar ele ser propagado.

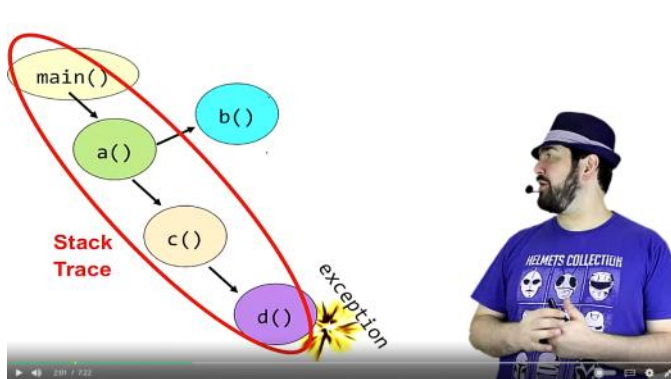
De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/fv3fD/hands-on-excecoes-na-pratica>>

# Stack Trace de Exceção

quinta-feira, 1 de junho de 2023 16:16

"Toda exceção deixa um rastro"

Toda vez que uma exceção é criada, ela grava a pilha de execução até aquele ponto. Essa informação pode ser muito útil para localizar erros e descobrir qual é o problema.



**A classe da exceção pode ajudar a identificar o tipo de erro que aconteceu!**

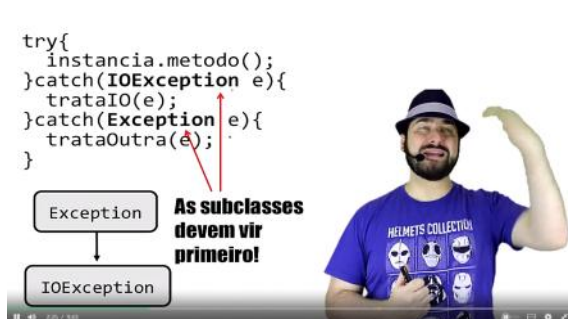
```
org.firebirdsql.jdbc.FBSQLException: Invalid column index.  
    at org.firebirdsql.jdbc.FBResultSet.getField(FBResultSet.java:617)  
    at org.firebirdsql.jdbc.FBResultSet.getField(FBResultSet.java:592)  
    at org.firebirdsql.jdbc.FBResultSet.getFloat(FBResultSet.java:496)  
    at com.myapplication.AccessServlet.doGet(AccessServlet.java:50)  
    at javax.servlet.http.HttpServlet.service(HttpServlet.java:689)  
    at javax.servlet.http.HttpServlet.service(HttpServlet.java:802)  
    at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:252)  
    at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:173)  
    at org.apache.catalina.core.StandardWrapperValve.invoke(StandardWrapperValve.java:213)  
    at org.apache.catalina.core.StandardContextValve.invoke(StandardContextValve.java:178)  
    at org.apache.catalina.core.StandardHostValve.invoke(StandardHostValve.java:126)  
    at org.apache.catalina.valves.ErrorReportValve.invoke(ErrorReportValve.java:105)  
    at org.apache.catalina.connector.CoyoteAdapter.service(CoyoteAdapter.java:148)  
    at org.apache.coyote.http11.Http11Processor.process(Http11Processor.java:869)  
    at org.apache.coyote.tomcat.net.PoolTcpEndpoint.processSocket(PoolTcpEndpoint.java:527)  
    at org.apache.tomcat.util.net.LeaderFollowerWorkerThread.run(LeaderFollowerWorkerThread.java:80)  
    at org.apache.tomcat.util.threads.ThreadPool$ControlRunnable.run(ThreadPool.java:684)  
    at java.lang.Thread.run(Thread.java:595)
```

# Tratando Exceções

quinta-feira, 1 de junho de 2023 16:16

Quando ocorre uma exceção, a execução "pula" para o bloco **catch** e continua depois dele.

Podem haver vários bloco **catch**, mas só um será executado.



```
try {
    // acessa banco
    // escreve arquivo
} catch (IOException | SQLException e) {
    tratarErro(e);
}
```

É possível declarar um mesmo tratamento de erro para mais de um tipo de exceção!

O bloco **finally** irá executar independente de acontecer a exceção ou não!

```
try {
    con = criaConexao();
    // acessa banco
    return algumaCoisa;
} catch(SQLException e) {
    trataErro(e);
} finally {
    con.close();
}
```

O **finally** é executado mesmo quando um valor é retornado!  
finally é **SEMPRE** executado!

```
try {
    a();
    b();
} catch (Exception e) {
    trataErro(e);
}
```

Observe que se ocorrer uma falha na execução de a(), a execução de b() não irá ocorrer.

```
try {
    a();
} catch(Exception e) {
    trateErro(e);
}
try {
    b();
} catch(Exception e) {
    trateErro(e);
}
```

```
}
```

Nesse caso, a execução de b() acontece independente de ocorrer um erro em a() ou não.

**Preste bem atenção onde você está colocando seus bloco de exceção!**

# Testando Exceções

quinta-feira, 1 de junho de 2023 16:16

Os cenários de teste precisam englobar também os casos em que acontecem erros!

Porque, na sua opinião, é importante testar casos em que exceções são lançadas pela classe? Facilitar a correção dos erros.

## Correto

As exceções devem ser lançadas quando erros ocorrem e, se elas não forem lançadas nessas situações, isso significa que o comportamento da classe está incorreto.

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/DqjNN/testando-excecoes>>

```
@Test(expected=BoomException.class)
```

```
public void testeErro() {  
    instancia.boom();  
}
```

Você pode declarar na anotação de teste que espera que uma exceção seja lançada.

O teste irá executar com sucesso se for lançada uma exceção em qualquer ponto do teste.

Preciso que de a exceção em algum lugar específico:

```
try{  
    instancia.boom();  
    fail();  
}catch(BoomException e){  
    //verifica informação  
    //da exceção  
}
```

**Quando a exceção precisa acontecer em um local específico ou suas informações precisam ser verificadas.**



Se um teste não espera uma exceção, apenas declare que o método joga a exceção.

```
@Teste
```

```
public void naoJogaExcecao() throws BoomExcpetion {  
    // este cenário não  
    // deve jogar exceção  
}
```

# Hands-on: Criando e Testando uma Classe que joga Exceções

quinta-feira, 1 de junho de 2023 16:16

## Projeto Login

Criação de uma classe que faz autenticação de usuário. Vai ocorrer erro caso o usuário não consiga fazer o login.

O que eu adiciono para fazer o teste do login falho, onde uma exceção precisa ser lançada para o comportamento estar correto?

Adiciono o atributo expected em @Test

**Correto**

Você deve configurar esse atributo com a exceção que espera que ocorra.

De <<https://www.coursera.org/learn/orientacao-a-objetos-com-java/lecture/sg0ed/hands-on-criando-e-testando-uma-classe-que-joga-excecoes>>